SC3020 Database System Principles
Project 2 Report

Group 7

Name	Matriculation No.	Contributions
Muhammad Firdaus Bin Azizan	U2121149J	Code, Sections 4.1
Mohamed Ali Mohamed Umar	U2121706D	Sections 3.3, 4.2
Wong Yi Xian	U2121401C	Code, Section 3.1
Cheong Yu Qing, Crystal	U2121124A	Code, Sections 1, 2.2, 3.2
Choo Yi Ken	U2240710B	Sections 2.1, 5

Table of Contents

1. Introduction **3**
1.1 Project Overview 3
1.2 Project Objectives 3
1.3 Dataset 3
2. System Design and Architecture **4**
2.1 Overview of the System Components 4
2.1.1 interface.py 4
2.1.2 whatif.py 5
2.1.3 preprocessing.py 5
2.1.4 project.py 7
2.1.5 Interactions between the modules 7
2.2 Flow of Execution 8
3. Key Algorithms **9**
3.1 Original QEP Retrieval Algorithm 9
3.2 Visualising the What-If Modifications 9
3.2.1 Key Considerations and Challenges 9
3.2.2 Algorithms 10
3.2.2.1 Parse and Construct the QEP Tree 10
3.2.2.2 Identify Target Node(s) for Modification 10
3.2.2.3 Apply Modifications 10
3.2.2.4 Propagate Changes 11
3.2.2.5 Validate Tree Consistency 11
3.2.2.6 Output Adjusted QEP 11
3.2.3 Maintaining Query Integrity 11
3.2.3.1 Dependency Validation 11
3.2.3.2 Cost and Performance Recalculation 12
3.2.3.3 Tree Consistency 12
3.2.3.4 User Verification 12
3.3 AQP Generation and Cost Comparison Algorithm 13
3.3.1 AQP Generation 13
3.3.2 Cost Comparison 14
4. Results **15**
4.1 Performance Evaluation 15
4.2 Example Query and Results 17
5. Limitations & Future Works **23**
5.1 Limited Scope of what-if analysis 23
5.2 Prolonged Execution Time for PostgreSQL 23
5.3 Limited Scalability for Complex Queries and Lack of User Interaction 23

1. Introduction

1.1 Project Overview

Efficient query execution is essential in relational databases, where the Query Execution Plan (QEP) determines how SQL queries are processed. The database optimizer selects the plan it estimates will have the lowest cost, but alternative query plans (AQPs) using different operators or join orders can sometimes yield better performance. Small changes, like switching from a hash join to a merge join or using an index scan instead of a sequential scan, can significantly impact execution time and resource usage.

Understanding these variations through what-if analysis is crucial for users seeking to optimise query performance. This project aims to develop a tool that allows users to visualise, modify, and compare query plans, providing insights into how execution strategies affect performance.

1.2 Project Objectives

The primary aim of this project is to create a tool that allows users to explore and analyse alternative query execution plans (AQPs) interactively. The system will help users understand the impact of different query execution strategies on query performance. The key objectives of the project are:

- **Visualise QEPs:** The system will retrieve the Query Execution Plan (QEP) of a given SQL query and display it in a visual tree format, making it easier to understand the query execution process for users.
- **Support what-if queries on the QEP:** Users will be able to modify physical operators or join orders in the visualised QEP to explore alternative execution strategies through what-if analysis.
- **Compare the cost of AQPs with the original QEP:** The system will generate alternative query plans (AQPs) based on user modifications and compare their estimated costs with the original QEP, allowing for performance insights.

1.3 Dataset

The dataset utilised is from The Transaction Processing Performance Council (TPC). The database consists of eight tables in total, namely `region.tbl`, `nation.tbl`, `part.tbl`, `supplier.tbl`, `partsupp.tbl`, `customer.tbl`, `orders.tbl`, `lineitem.tbl`, which were used in implementing our project. The database is obtained from the following URL:
https://www.tpc.org/tpc_documents_current_versions/current_specifications5.asp

2. System Design and Architecture

2.1 Overview of the System Components

This section will focus on a detailed explanation of the program files used in our project: `interface.py`, `whatif.py`, `preprocessing.py`, and `project.py`.

2.1.1 `interface.py`

Using the Streamlit module, the `interface.py` module provides the user interface for interacting with SQL QEPs. First, users can input their intended SQL query and click a button to add the query to the list of QEPs. Then, the system will check for the query. If the query is invalid or has already existed in the query list, error messages will pop up, indicating that the user should change the query. Else, the query is added to the query list in session state with a unique key and a success message will pop up. After that, the original QEP is generated using `get_query_plan()` function and alternative QEPs (AQPs) are generated using `get_all_query_plan()` function.

To view the performance of queries by comparing original QEP and corresponding AQPs, users can first choose a query to be processed from the dropdown box. After a query is being selected, the original QEP is retrieved using `state.original_qep.get()` and alternative QEP is retrieved upon user's selection using `qep_settings.get()`.

Both queries are rendered using `render_query()` function and displayed side-by-side for comparison in textual content and graphical representations. On the left pane, the original QEP is rendered and the alternative QEP is rendered on the right pane. On both panes, a list of steps to build the QEP tree is listed by using `get_qep_steps_to_str()`, `build_tree_and_get_steps()` functions. Also, both query execution trees are displayed at the bottom of the steps using images.

The interface is systematically partitioned into four key components—`input_section()`, `selection_section()`, `metrics_section()`, and `render_section()`—each playing a distinct role in guiding users through the query analysis process. This modular workflow ensures that users can efficiently input, validate, select, and analyse SQL queries, making the interface an intuitive and powerful tool for database performance optimization.

Main functions:

- `get_generator()`: Initializes and returns an instance of `QueryPlanAnalyzer` for processing
- `query_generator()`: Retrieves the query plan for the given query
- `get_explanation_graph()`: Creates a graphical representation of the QEP for visualisation
- `render_query()`: Displays the original and alternative QEPs side-by-side for comparison
- `input_section()`: Handles user input for SQL queries, validates them, and generates QEPs
- `metrics_section()`: Displays key performance metrics for the selected query
- `selection_section ()`: Allows users to select and view previously submitted queries
- `render_section()`: Renders QEP visualisations for the selected query and its settings
- `main()`: Entry point for Streamlit application

2.1.2 `whatif.py`

As the name suggests, the `whatif.py` module is in charge of exploring and visualising alternative execution scenarios for SQL queries, rendering other possibilities ('whatif' scenarios) to be executed. Using this module, users are able to view what will happen if the query planner chooses another execution strategy which is different from our original QEP, for example: what if a hash join is used instead of a block-based nested loop join or what if a sequential scan is used instead of an index scan. By simulating different execution methods for the same query, users can analyse trade-offs between execution strategies and understand why the query planner chooses certain approaches under specific conditions.

Graphical representations of QEP and AQP trees are also built using this module by build_qep_tree() function which takes in a .json format data of the QEP and returns the root node of the QEP tree, representing the generated tree. The execution plan trees consist of interconnected Node objects, where each node represents a specific step or operation in the query execution process, such as scans, joins, aggregations, or sorts. Steps regarding the execution flow are retrieved using the get_qep_steps_to_str() function which takes the root node of a QEP tree, and outputs the corresponding message string depending on the type of root node passed in (node.node_type).

Additionally, performance metrics such as the number of shared buffers accessed during execution (`node.node_buffer`), the number of rows processed at each step (`node.row_number`), the total time spent on the execution step (`node.node_time`), and the estimated cost of the execution step (`node.node_total_cost`) are also appended to the message string above, alongside with the graphical representation. These metrics provide users with critical insights into the performance of each execution step, allowing users to compare scenarios and identify the most efficient execution strategy.

Main classes:

- `Node` => The tree nodes in QEP tree
  - `add_child()`: Takes a child node as input and adds it to the list of children
  - `num_children()`: Returns the number of children for this node
  - `get_qep_steps()`: Generates the QEP steps in bottom up manner

Main functions:

- `build_qep_tree()`: Takes QEP in json as input and generates a tree structure for the QEP
- `build_tree_and_get_steps()`: Takes the root node of a QEP tree and generates nodes for the QEP visualisation
- `get_qep_steps_to_str()`: Returns the query execution plan steps in string format
- `build_graph()`: Build graph representation from QEP reasoning and return image bytes

2.1.3 `preprocessing.py`

The `preprocessing.py` module handles most of the backend tasks that are required such as connecting PostgreSQL server, quick query validation, transformation, and execution plan retrieval, which serves as the backbone of the project. Before generating the QEPs, a quick check of the validity of the query is performed using the `try_execute()` function after establishing connection with the server.

One of the key features in the `preprocessing.py` module is the integration of the Current Tuple ID **(CTID)** into SQL queries using `_transform_query()` function in `SQLQueryTransformer` class, offering deeper insights into data access patterns. CTID, a system column in PostgreSQL, uniquely identifies the physical location of a row within a table based on its storage block and offset. This feature is primarily utilised in the `get_block_access()` function, which leverages CTID to analyse and retrieve the specific blocks accessed during query execution. By transforming SQL queries to include CTID columns, the function identifies the block numbers for accessed rows and retrieves their full content, grouping results by block ID. This organised block-level data enables granular analysis of data access patterns, helping to identify performance bottlenecks and uncover opportunities for optimization. Through CTID, the function provides a robust tool for understanding query behaviour at the physical storage level, offering critical insights for database performance tuning.

Another key feature of the `preprocessing.py` module is the generation of QEPs using the `QueryPlanAnalyzer` class. This is done by the `get_query_plan()` function which retrieves QEPs in `.json` format using the `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` command. Along with the QEP, key information such as node types, join and filter conditions, row estimates, and actual runtime performance metrics are also retrieved together. Alternative QEPs (AQPs) are also retrieved using `get_query_plan_with_settings()` and `get_all_query_plans()` functions according to users' customisations. This enables direct comparisons of performance metrics between QEPs and AQPs, facilitating the analysis and optimization of different execution strategies.

Main classes:

- `SQLQueryTransformer` => Transforms a given SQL query to include CTID of accessed blocks
  - `_format_query()`: Formats the SQL query to a standardised structure
  - `_get_subqueries()`: Extracts subqueries from the SQL query
  - `_get_columns()`: Extracts columns from a SELECT or GROUP BY clause
  - `_has_aggregate()`: Checks if the query contains aggregate functions
  - `_transform_query()`: Transforms a query by adding CTID columns
  - `_process_nested_query()`: Processes nested queries recursively to add CTID columns

- `DatabaseConnection` => Represents a connection to the database
  - `try_execute()`: Quick check if the query is valid
  - `close()`: Closes the connection to the DB

- `QueryExecutionPlan` => Represents the query execution plan details

- `QueryPlanAnalyzer` => Provides functionalities to analyse and fetch query execution plans.
  - `_enable_execution_settings()`: Sets the necessary database settings for query execution analysis
  - `set_execution_settings()`: Sets specific PostgreSQL execution settings for query analysis
  - `check_query()`: Checks if the query is valid
  - `get_query_plan()`: Returns the query execution plan for the given query

- `get_query_plan_with_settings()`: Executes the query with specific execution settings and returns the query plan
- `index_present()`: Checks if any indexes exist on the specified table
- `get_all_query_plans()`: Generates query execution plans with different settings
- `get_block_access()`: Returns the full content of accessed blocks for the given query

Other functions:

- `transform_query()`: Provides a dropdown box for selecting a query

2.1.4 `project.py`

The project.py file serves as a simple entry point for our project. Upon execution, the main() function from the interface.py file is called which is in charge of initialising the Streamlit interface and managing user inputs, session states and QEP comparisons.

2.1.5 Interactions between the modules

The project is built on a modular architecture where each module handles specific functionalities, interacting seamlessly to generate and visualise Query Execution Plans (QEPs). The `interface.py` module serves as the UI component, allowing users to input SQL queries and interact with the system. It relies on `preprocessing.py` for backend logic, such as query validation, transformation, and analysis. For instance, `interface.py` uses the `QueryPlanAnalyzer` class from `preprocessing.py` to validate queries, retrieve QEPs, and process adjustable execution plans based on user-defined settings. Functions like `try_execute()`, `get_query_plan()`, and `get_all_query_plans()` enable this interaction, ensuring smooth data flow from user input to backend processing.

The `interface.py` module also interacts with `whatif.py` to provide users with detailed QEP reasoning and visualisation. It leverages functions such as `build_qep_tree()` and `get_qep_steps_to_str()` to transform the QEP data into structured insights and human-readable reasoning. Additionally, the `build_graph()` function in `whatif.py` generates graphical representations of the QEPs, which are displayed in the UI. These visualisations help users understand query performance and optimise execution strategies effectively. The data consistency between `preprocessing.py` and `whatif.py` is crucial, as the processed QEP structure is shared seamlessly for reasoning and visualisation tasks.

Furthermore, `project.py` serves as the entry point for the application, initialising and running the UI through `interface.py`. This separation of concerns ensures that `project.py` focuses solely on application startup and runtime configuration, leaving all interface logic to `interface.py`. The modular interactions facilitate a clear data flow, where SQL queries from the user are validated and processed by `preprocessing.py`, then passed to `whatif.py` for reasoning and visualisation before being presented back to the user via the UI.

2.2 Flow of Execution

1. Parse the Query Execution Plan (QEP):

The QEP in JSON format is parsed to extract plan attributes and hierarchical relationships using the `build_qep_tree()` function. Each node in the plan (e.g., joins, scans, sorts) is represented as a Node object with attributes like cost, buffer size, and join strategy.

2. Construct the QEP Tree:

A tree structure is built, where each Node represents an operation in the QEP, and its children represent dependent sub-operations.

3. Apply Adjustments:

Traversal functions (`build_tree_and_get_steps()`) are used to navigate the tree and identify the node to modify. Specific attributes of the node (e.g., `node_type, hash_condition, merge_condition`) are updated based on user input. For example: Changing a Hash Join to a Merge Join involves modifying `node_type` and updating the `merge_condition`.

4. Recompute Derived Properties:

After applying changes, related attributes (e.g., `node_cost, node_total_cost`) are recalculated to reflect the updated execution plan.

5. Visualise Adjusted Plan:

The modified tree is converted back into traversal steps or a graphical representation for user verification.

3. Key Algorithms

3.1 Original QEP Retrieval Algorithm

When the system receives the query entered by the user, it first checks if the syntax and structure of the query are valid using the `is_valid_query(query)` function. The function uses the "`PREPARE`" keyword to create a query plan without running it. This ensures that the syntax and structure are valid before the actual execution of the query. If the syntax is incorrect, the query plan will not be prepared and the function will raise an exception to inform the user. If the preparation succeeds, it then deallocates memory for the plan. Lastly, the database will be rolled back to undo any uncommitted changes during the database session, preventing any side effects from failed query preparations.

Once the query is error-free, the application proceeds to retrieve the optimised query execution plan for the user-stated query using the `get_query_plan()` function. This function uses the "`EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`" statement to retrieve the optimised query plan along with other statistics such as the runtime performance of the query and total buffer usage during query execution. This statement then returns the QEP in JSON format for easier data manipulation.

Once the optimised query plan is retrieved, a tree structure is built to display the QEP in a hierarchical format. The tree-building process is iterative, where each plan is extracted from the QEP and is represented using a node structure. Each node will contain information about each plan, such as node type, node cost, or any join condition involved. Each newly created node is then added to a list of parent nodes to showcase the parent-child relationships between each plan. The first plan extracted will act as the root node of the tree and the next plan will be added as its child node and therefore recursively build the tree structure for a QEP.

3.2 Visualising the What-If Modifications

This section outlines the methodology for applying adjustments to a Query Execution Plan (QEP) tree to simulate alternate execution strategies, operators, or configurations. The algorithm ensures logical consistency and correctness while enabling modifications such as switching operator types (e.g., `"Hash Join"` to `"Merge Join"`) or adjusting associated costs and attributes.

3.2.1 Key Considerations and Challenges

Logical Integrity: Changes to the QEP tree must preserve the logical requirements of the query. For example, replacing a `Hash Join` with a `Merge Join` necessitates the presence of sorted inputs. The algorithm dynamically introduces a Sort operation if needed.

Recalculation of Costs: Adjustments impact both local (`node_cost`) and cumulative (`node_total_cost`) costs. The algorithm ensures that:

- Node costs are recalculated to reflect the new operator's characteristics.
- Cost changes propagate upward to maintain the accuracy of the entire tree.

3.2.2 Algorithms

3.2.2.1 Parse and Construct the QEP Tree

The QEP is provided in a hierarchical format (e.g., JSON). The algorithm begins by parsing the plan using the `build_qep_tree()` function, which creates a tree structure where:

- **Nodes:** Each node represents a specific operation (e.g., `Seq Scan`, `Hash Join`).
- **Attributes:** Nodes store metadata such as `node_cost`, `relation_name`, and join conditions.
- **Parent-Child Relationships:** Hierarchical dependencies are established, where the root node represents the top-level operation, and leaf nodes correspond to base table scans.

The tree structure facilitates traversal and modification of individual nodes, supporting efficient adjustments.

3.2.2.2 Identify Target Node(s) for Modification

This step ensures precise targeting of the desired node(s) while preserving the overall tree structure. Traversal algorithms are employed to locate specific nodes in the QEP tree for modification. The identification process involves:

- **Type-Based Filtering:** Nodes are matched based on their `node_type` attribute (e.g., locating all `Hash Join` nodes).
- **Attribute-Based Filtering:** Additional filters are applied to identify nodes based on specific attributes, such as `relation_name` or `hash_condition`.

3.2.2.3 Apply Modifications

Once the target node is identified, its attributes are updated to reflect the desired adjustment. Common modifications include:

- **Operator Type Changes:** Updating the `node_type` attribute to change the operation (e.g., from `Hash Join` to `Merge Join`).
- **Condition Updates:** Modifying associated conditions (e.g., replacing `hash_condition` with `merge_condition`).
- **Insertion of Prerequisite Nodes:** Adding intermediate operations such as a `Sort` node when required by the new operator (e.g., ensuring sorted input for a `Merge Join`).
- **Cost Adjustments:** Recalculating `node_cost` and `node_total_cost` to account for the new operation's characteristics.

Example: To change a `Hash Join` to a `Merge Join`:

1. Update `node_type` to `"Merge Join"`.
2. Replace `hash_condition` with `merge_condition`.
3. Ensure a `Sort` node is present in the subtree to provide ordered inputs.

3.2.2.4 Propagate Changes

Adjustments made to a node may impact its parent and sibling nodes. To ensure consistency, the algorithm propagates changes upward in the tree, recalculating costs and updating dependencies:

- **Node Costs:** Recompute the cost of the modified node based on its new configuration.
- **Cumulative Costs:** Update the `node_total_cost` of parent nodes to reflect the adjusted subtree.

3.2.2.5 Validate Tree Consistency

After applying modifications, the tree is validated to ensure logical and structural consistency:

Dependency Checks: Confirm that required prerequisites for the modified operator are satisfied (e.g., verifying sorted inputs for a `Merge Join`).

Structural Integrity: Ensure that parent-child relationships remain intact and that all nodes are properly connected.

3.2.2.6 Output Adjusted QEP

The modified QEP tree is converted into:

- **Step-by-Step Plan:**
  A linear sequence of operations derived through a bottom-up traversal of the tree.
- **Graphical Representation:**
  A visual depiction of the adjusted plan, generated using tools such as `pygraphviz`.

These outputs allow users to review the modified execution plan and verify its correctness before implementation.

3.2.3 Maintaining Query Integrity

Ensuring the integrity of a modified Query Execution Plan (QEP) is critical to preserving the correctness and logical soundness of the query. The system incorporates several mechanisms to validate and maintain the consistency of the modified plan, ensuring it aligns with the original query's intent and can be executed without errors.

3.2.3.1 Dependency Validation

Changes to specific operations within the QEP, such as switching from a `Hash Join` to a `Merge Join`, may introduce new dependencies that must be satisfied to maintain query correctness. Dependency validation involves:

Ensuring Prerequisite Operations: Operators such as `Merge Join` require sorted input data. If sorting is not already performed in the existing QEP, the system automatically inserts a `Sort` node in the appropriate location within the tree.

Maintaining Valid Conditions: The system updates and validates conditions associated with the modified operation (e.g., replacing `hash_condition` with `merge_condition` for a `Merge Join`) to ensure they remain logically consistent with the original query semantics.

This validation guarantees that the QEP adheres to the logical requirements of each operator after modifications.

3.2.3.2 Cost and Performance Recalculation

Following modifications, the system recalculates both local and cumulative costs associated with the affected nodes to preserve the integrity of performance metrics. This includes:

Node Cost Adjustment: The cost of the modified node (`node_cost`) is recalculated based on the characteristics of the new operation (e.g., the cost of a `Merge Join` versus a `Hash Join`).

Cumulative Cost Propagation: The `node_total_cost`, which represents the cumulative cost of executing the subtree rooted at a node, is recomputed and propagated upward through the tree. The formula for cumulative cost calculation is: $node_total_cost = node_cost + \sum(child_node.node_total_cost)$

This recalibration ensures the performance estimates for the modified QEP remain accurate.

3.2.3.3 Tree Consistency

Structural consistency of the QEP tree is crucial to ensure the integrity of the modified plan. The system performs the following checks:

Parent-Child Relationship Validation: The hierarchical structure of the tree is verified to ensure all parent-child relationships are intact and correctly represent the dependencies among operations.

Alignment with Modified Plan: Each node's operation is validated against the modified QEP to ensure compatibility. For example, if a `Merge Join` is introduced, the system checks for the presence of a preceding `Sort` operation to provide ordered inputs.

These validations ensure the tree structure is logically sound and executable after modifications.

3.2.3.4 User Verification

To enhance reliability, the system presents the adjusted QEP to the user for manual review. This is achieved through:

Step-by-Step Plan Representation: The modified QEP is converted into a sequence of textual steps, detailing each operation and its associated attributes, enabling users to trace and verify the execution flow.

Graphical Visualisation: The QEP is rendered as a graph, providing an intuitive visual representation of the modified execution plan and its hierarchical structure.

3.3 AQP Generation and Cost Comparison Algorithm

3.3.1 AQP Generation

Next, to handle the various what-if scenarios, the application also retrieves various alternative query plans based on multiple database settings. This is done through the `get_all_query_plans()` method, which is in preprocessing.py. This method does not directly take the modified QEP and generate the corresponding AQP, but rather it modifies how the database will execute the query itself, based on different execution settings related to join strategies and scanning methods. The settings applied will then influence the query plan that is generated based on the what-if settings.

In short, in the user interface, the user will input the query that will generate a QEP and select a setting for the Adjustable QEP which will then use the method to influence how the database generates the plan, to generate the AQP.

Let's discuss this with an alternate query plan setting, *'Hash Join with Index Scan'*. This setting is used to generate the corresponding alternative plan of a given query with Hash Join and Index scan and not allow other join-types or scan-types to be in the alternative plan. This is done through the following steps:

1. **Check for Join Type in Query:** Using `checking_join_type()`, the method determines if the query contains any joins. It searches for explicit JOIN keywords or implicit WHERE keywords. If `checking_join_type()` returns True, it indicates the query involves a join.

2. **Generate Settings:** Here, the *possible_setting* is generated based on the user settings and the checking for join types. Here, the setting *'Hash Join with Index Scan'* is used to see how the *possible_setting* is generated.

    a. If `checking_join_type()` returns True, the following entry in *possible_settings* is configured:
        - Hash Join (*enable_hashjoin*: 'on')
        - Index Scan(*enable_indexscan*: 'on')
        - Disable Merge Join, Nested Loop Join, and Sequential Scan by setting their corresponding flags to 'off'

    b. If `checking_join_type()` returns False, the entry in *possible_settings* will just omit the join settings

3. **Apply Configurations:** In each of the configurations in the previous step, the settings are passed to `set_execution_settings()`, which adjusts the execution strategy of the database to enable certain joins and scans while disabling others. In this case, the strategy will include Hash Join and Index Scan while disabling other join and scan types.

4. **Generate Alternate Plan:** With the applied configurations, `get_query_plan()` is called to retrieve the query plan with the alternate configurations. In this case, the database query optimizer will only use Hash Join and Index Scan for the AQP.

3.3.2 Cost Comparison

After the QEP and AQP are generated, the results are displayed in the form of graphs, steps, and costs related to each step. The costs related to the plans are obtained and calculated in the `build_qep_tree()` method which is located in the what-if.py where the costs are extracted and calculated. This is done through the following steps:

1. **Extraction of Data**: The QEP or the AQP is generated by the database in the form of a JSON structure. This structure contains various fields, which the following fields are used for the cost and cost calculation:
    - *"Actual Total Time"* - Total execution time for a node which includes startup and processing
    - *"Actual Startup Time"* - Time taken before processing the rows
    - *"Total Cost"* - Estimated cost of the node which includes both startup and processing
    - *"Plan Rows"* - Estimated rows the node will process
    - *"Shared Hit Block"* - Number of blocks read from the memory

2. **Calculation**: After extracting the above data from the JSON structure, All of these costs are used directly from the extracted data. The above data are stored or calculated as follows:
    - *node_total_cost* ← *Total Cost*
    - *node_time* ← *Actual Total Time*
    - *node_buffer* ← *Shared Hit Blocks*
    - *row_number*← *Plan Rows*

3. **Display**: The above variables are used to build an object *node* that represents the query execution plan node. This node contains the relevant cost-related attributes that will be used to display the cost values in a desired way which is structured in the `get_qep_steps_to_str()`. For each node in an execution plan, the costs are displayed as follows:
    - Buffer Hit: <*node_buffer*> | Plan Rows: <*row_number*> | Total Time: <*node_time*> | Total Cost: <*node_total_cost*>

After generating the Costs, comparison can be done easily by comparing each node of the QEP and its AQP by comparing its cost attributes.

4. Results

4.1 Performance Evaluation

For the Query Execution Plan (QEP) retrieval, the QueryPlanAnalyzer class in preprocessing.py generates the original QEP using PostgreSQL's EXPLAIN command. This process includes both Planning_Time and Execution_Time, providing insights into the database's processing time. The get_all_query_plans method applies various PostgreSQL planner settings to generate Adaptive Query Plans (AQPs), allowing comparisons of planning and execution times across settings.

To modify and visualise AQPs, the `whatif.py` module uses functions like `build_tree_and_get_steps()` and `build_graph()`. These functions process the QEP, creating tree structures and visual representations that detail costs, times, and metrics such as buffer hits for each node in the execution plan.

Performance tracking involves logging Planning_Time and Execution_Time attributes returned by `get_query_plan()` and `get_query_plan_with_settings()`. These metrics are evaluated across different planner settings to determine the performance impact. QEP generation is heavily dependent on query complexity, with joins and nested queries increasing planning times.

Time Taken for QEP Retrieval and Modification, retrieving QEPs for simple queries is efficient, but complex queries with joins or aggregates may take significantly longer.

Let's compare a simple query to a complex query to see how much longer it will actually take to retrieve QEP [Planning Details (Plan Buffer, Plan Time, Execution Time) are metrics associated with retrieving the Query Execution Plan (QEP) and the subsequent execution of the query]:

Simple Query	Complex Query

`SELECT`
`  r.r_name AS region_name,`
`  COUNT(n.n_nationkey) AS nation_count`
`FROM`
`  public.region r`
`LEFT JOIN`
`  public.nation n`
`ON`
`  r.r_regionkey = n.r_regionkey`
`WHERE`
`  r.r_regionkey = 1`
`GROUP BY`
`  r.r_name;`
	`SELECT`
	`  r.r_name AS region_name,`
	`  n.n_name AS nation_name,`
	`  COUNT(DISTINCT o.o_orderkey) AS total_orders,`
	`  SUM(l.l_extendedprice * (1 - l.l_discount)) AS revenue`
	`FROM`
	`  public.region r`
	`JOIN`
	`  public.nation n`
	`ON`
	`  r.r_regionkey = n.r_regionkey`
	`JOIN`
	`  public.customer c`
	`ON`
	`  n.n_nationkey = c.c_nationkey`
	`JOIN`

	`public.orders o`
	`ON`
	`c.c_custkey = o.o_custkey`
	`JOIN`
	`public.lineitem l`
	`ON`
	`o.o_orderkey = l.l_orderkey`
	`WHERE`
	`r.r_regionkey = 1`
	`AND l.l_shipdate BETWEEN '2024-01-01'`
	`AND '2024-12-31'`
	`GROUP BY`
	`r.r_name, n.n_name`
	`ORDER BY`
	`revenue DESC;`

A single join and aggregation, low computational cost.	Multiple joins, filters, and aggregate functions on large datasets, resulting in higher computational cost.
Plan Buffer: 194 Plan Time: 1.194 ms Execution Time: 0.085 ms	Plan Buffer: 110 Plan Time: 4.632 ms Execution Time: 658.624 ms

The simple query involves only a single join and aggregation, resulting in lower planning and execution times. In contrast, the complex query requires multiple joins, filters, and aggregations across larger datasets. This leads to significantly higher computational cost, as evidenced by the increased Plan Time (4.632 ms) and Execution Time (658.624 ms).

Metric	Simple Query	Complex Query
Plan Buffer	194	110
Plan Time (ms)	1.194	4.632
Execution Time (ms)	0.085	658.624

These results demonstrate that the algorithm efficiently retrieves and modifies QEPs for simple queries, but as query complexity increases, planning and execution costs rise significantly. This highlights the importance of optimising query structures and planner settings to minimise performance overhead.

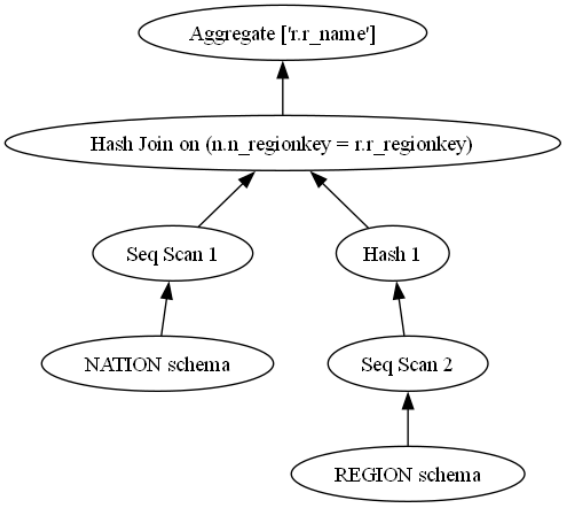
4.2 Example Query and Results

This section will focus on comparing the original QEP to their corresponding AQPs.In this section, an example query will be used to generate an optimised query plan, as well as its alternative query plans based on the what-if settings. The QEP and its corresponding AQPs generated will be compared in terms of costs to derive insights into the various what-if scenarios of the QEP.

Original Example Query

```
SELECT
   r.r_name AS region_name,
   COUNT(n.n_nationkey) AS nation_count
FROM
   public.region r
JOIN
   public.nation n
ON
   r.r_regionkey = n.r_regionkey
GROUP BY
   r.r_name
```

Optimised QEP - Hash Join and Sequential Scan



Step 1: Groups up/Aggregates attribute r_name from table R using the Hashed strategy.
Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.04 ms | Total Cost: 28.53

Step 2: N is joined with R using the Hash Join method by comparing attribute n_regionkey from N and r_regionkey from R.
Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.03 ms | Total Cost: 25.98

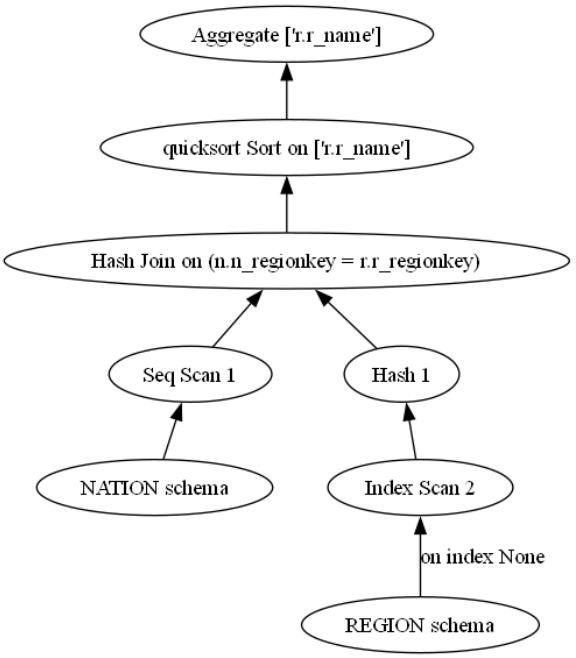
Step 3: Do a Sequential Scan on table NATION.
Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 11.70

Step 4: Hash NATION's buckets.
Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 11.70

Step 5: Do a Sequential Scan on table REGION.
Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 11.70

Insights: The optimised QEP uses hash join and sequential scan, resulting in the most efficient out of all the query plans in terms of cost and buffer hits.

AQP - Hash Join with Index Scan



Step 1: Groups up/Aggregates attribute r_name from table R using the Sorted strategy.
Buffer Hit: 6 | Plan Rows: 170 | Total Time: 0.05 ms | Total Cost: 10000000074.25

Step 2: R table is sorted using QUICKSORT method according to attributes r_name in a ASCENDING order.
Buffer Hit: 6 | Rows: 170 | Total Time: 0.05 ms | Total Cost: 10000000071.70

Step 3: N is joined with R using the Hash Join method by comparing attribute n_regionkey from N and r_regionkey from R.
Buffer Hit: 3 | Plan Rows: 170 | Total Time: 0.03 ms | Total Cost: 10000000064.98

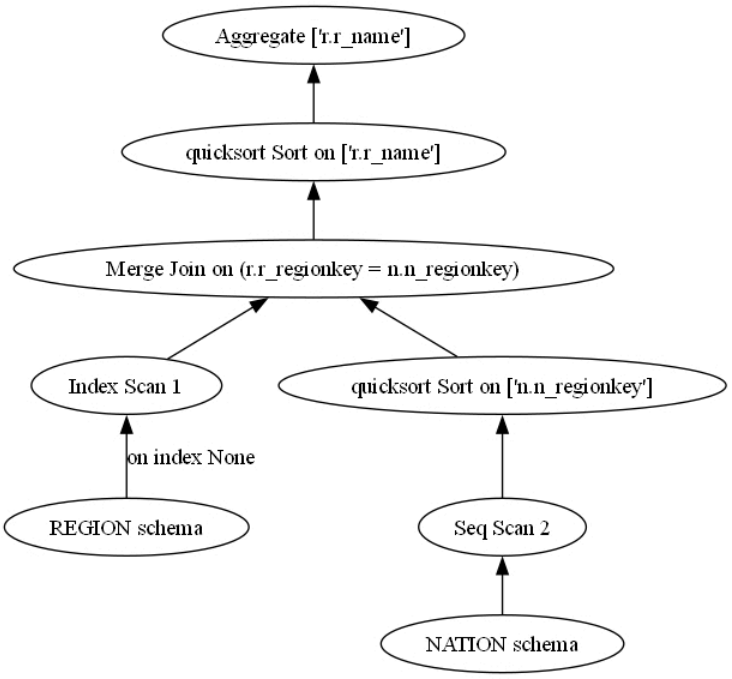
Step 4: Do a Sequential Scan on table NATION.
Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 10000000011.70

Step 5: Hash NATION's buckets.
Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 50.70

Step 6: Do an Index Scan on table REGION on index name region_pkey.
Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 50.70

Insights: This AQP uses hash join with index scan. Adding an index scan on REGION led to a significant increase in the total cost, possibly due to the sorting requirements. Compared with the original QEP, the replacement of a sequential scan with the index scan added complexity to the query plan.

AQP - Merge Join with Index Scan



Step 1: Groups up/Aggregates attribute r_name from table R using the Sorted strategy.
Buffer Hit: 6 | Plan Rows: 170 | Total Time: 0.05 ms | Total Cost: 10000000074.25

Step 2: R table is sorted using QUICKSORT method according to attributes r_name in a ASCENDING order.
Buffer Hit: 6 | Rows: 170 | Total Time: 0.05 ms | Total Cost: 10000000071.70

Step 3: N is joined with R using the Hash Join method by comparing attribute n_regionkey from N and r_regionkey from R.
Buffer Hit: 3 | Plan Rows: 170 | Total Time: 0.03 ms | Total Cost: 10000000064.98

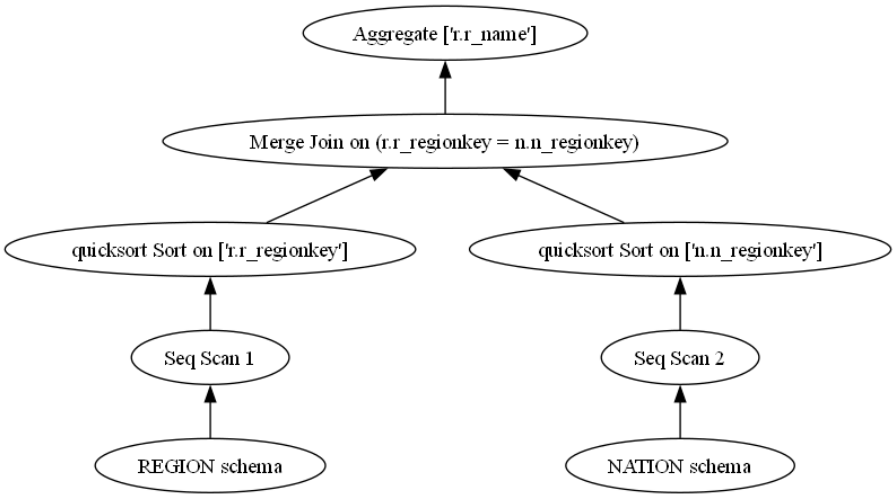
Step 4: Do a Sequential Scan on table NATION.
Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 10000000011.70

Step 5: Hash NATION's buckets.
Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 50.70

Step 6: Do an Index Scan on table REGION on index name region_pkey.
Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 50.70

Insights: This AQP replaces both the scan type and the join type of the optimised QEP by using Merge-join and index scan. This also leads to very high costs. This may be because merge join requires a sorted input, resulting in the formation of Quick Sort. Therefore, with the additional cost of sorting and indexing, it is far more costlier than the optimised QEP.

AQP - Merge Join with Sequential Scan



Step 1: Groups up/Aggregates attribute r_name from table R using the Hashed strategy.
Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.03 ms | Total Cost: 41.95

Step 2: R is joined with N using the Merge Join method by comparing attribute r_regionkey from R and n_regionkey from N.
Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.02 ms | Total Cost: 39.40

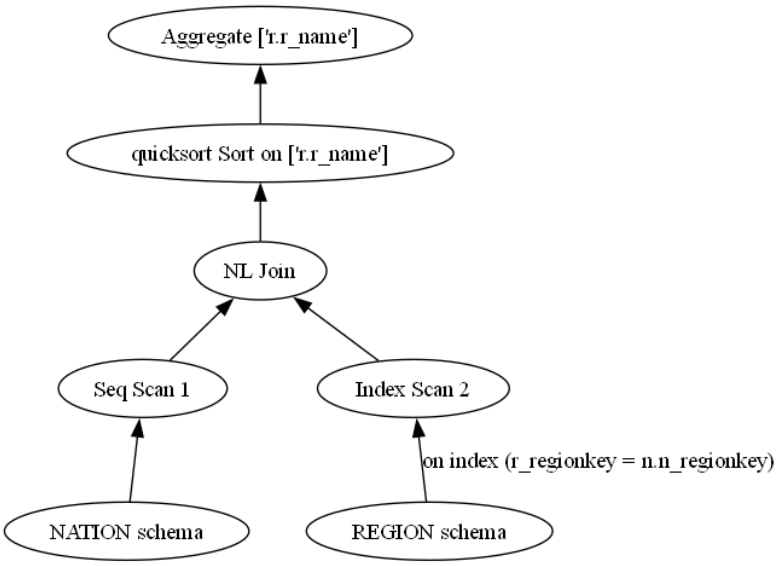
Step 3: R table is sorted using QUICKSORT method according to attributes r_regionkey in a ASCENDING order.
Buffer Hit: 1 | Rows: 170 | Total Time: 0.01 ms | Total Cost: 18.42

Step 4: Do a Sequential Scan on table REGION.
Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 11.70

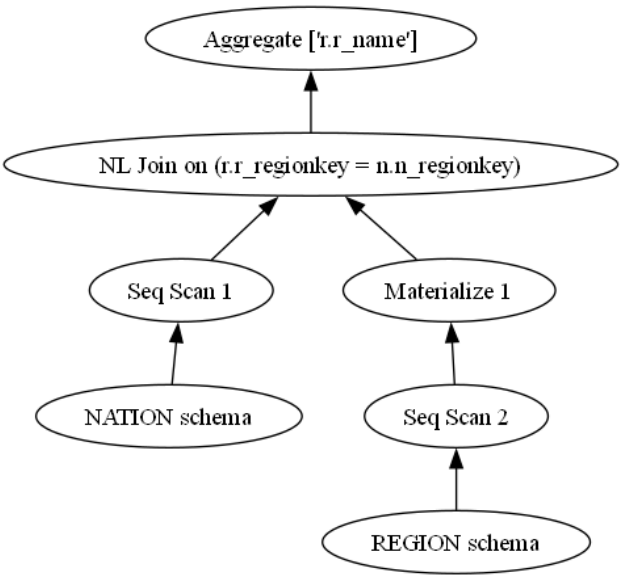
Step 5: N table is sorted using QUICKSORT method according to attributes n_regionkey in a ASCENDING order.
Buffer Hit: 1 | Rows: 170 | Total Time: 0.01 ms | Total Cost: 18.42

Step 6: Do a Sequential Scan on table NATION.
Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 11.70

Insights: This AQP replaces the join type with merge join and uses sequential scans on the join keys resulting in a moderate total cost. Here, it can be seen that sequential scan does decrease reading costs, but the additional sorting steps will still increase the overhead. This plan, although more efficient than the previous AQP, still falls short of the optimised plan due to the added cost of sorting.

AQP - Nested Loop with Index Scan

Step 1: Groups up/Aggregates attribute r_name from table R using the Sorted strategy. Buffer Hit: 51 | Plan Rows: 170 | Total Time: 0.04 ms | Total Cost: 10000000096.30 Step 2: R table is sorted using QUICKSORT method according to attributes r_name in a ASCENDING order. Buffer Hit: 51 | Rows: 170 | Total Time: 0.04 ms | Total Cost: 10000000093.75 Step 3: NATION is joined with REGION using the Nested Loop method. Buffer Hit: 51 | Plan Rows: 170 | Total Time: 0.03 ms | Total Cost: 10000000087.03 Step 4: Do a Sequential Scan on table NATION. Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 10000000011.70 Step 5: Do an Index Scan on table REGION on index name region_pkey. Buffer Hit: 50 | Plan Rows: 1 | Total Time: 0.00 ms | Total Cost: 0.44

Insights: In this AQP, Nested Loop join and index scan are used, which resulted in higher buffer hits and costs. Here, the index scan on REGION increased buffer hits significantly. Hence, in comparison with the optimised query plan, a nested loop join may not be suitable.

AQP - Nested Loop with Sequential Scan

Step 1: Groups up/Aggregates attribute r_name from table R using the Hashed strategy. Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.03 ms | Total Cost: 457.35 Step 2: REGION is joined with N using the Nested Loop method by comparing the attribute r.r_regionkey from REGION and n_regionkey from N. Buffer Hit: 2 | Plan Rows: 170 | Total Time: 0.02 ms | Total Cost: 454.80 Step 3: Do a Sequential Scan on table NATION. Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.01 ms | Total Cost: 11.70 Step 4: Do a Sequential Scan on table REGION. Buffer Hit: 1 | Plan Rows: 170 | Total Time: 0.00 ms | Total Cost: 11.70

Insights: In this AQP, sequential scan with nested loop still incurs high costs. This may be because, without an index, nested loops are inefficient for joins. However, this AQP still produces a significantly lower cost than the previous AQP due to the use of sequential scans. However, this approach is still less efficient than the optimised QEP.

Overall Insights

For this example query, the original optimised query remains the most efficient, with low cost, minimal buffer hits, and fast execution time due to the use of hash join with sequential scan and no sorting.

AQPs that used an index scan instead of the sequential scan, saw a significant overhead due to sorting requirements, leading to high cost and buffer hits without improving the performance significantly. The use of nested loop joins for this query is still less efficient especially when using an index scan, which greatly increases buffer hits and costs.

The closest alternative in terms of cost-efficiency is the AQP with merge join and sequential scan, which is slightly more costlier than the optimised QEP due to the sorting requirements.

5. Limitations & Future Works

5.1 Limited Scope of what-if analysis

The current implementation of our tool primarily focuses on modifications towards a limited set of queries such as switching the join algorithm (e.g. hash join and sort-merge join) or changing the join orders of tables.

However, in the problem of query optimisation, there are still vast issues that may be influential towards our performance that need to be addressed. These issues include the usage and creation of indexes on tables and analysis of AQP formulations through query rewriting mechanisms. Thus, this limited scope restricts the tool's applicability for complex, real-world database optimization tasks. It may also fail to provide users with a holistic understanding of how various factors interact in query optimization in real world scenarios.

5.2 Prolonged Execution Time for PostgreSQL

In our project implementation, one of the notable findings is that the execution time for complex queries is significantly high. Query complexity is mainly determined by a few factors including the number of joins across large tables, nested subqueries and presence of aggregation and sorting clauses which require PostgreSQL to process the whole table for processing, which PostgreSQL will find it hard to maintain optimal performance. To counter this issue, there are several measures that can be taken into consideration:

- Additional clauses such as `LIMIT` can be introduced into our query to limit the output size to prevent overloading and crashing the system.
- Apply filters to rows using WHERE clauses to also limit the intermediate output size so that less amount of data is processed by PostgreSQL.
- Join orders can also be optimised by using left-deep join trees and ensuring smaller tables are joined first in the left part of the tree.
- Consider removing ORDER BY clauses when sorting is not critical in the implementation as scanning through the table in either one-pass or multiple-passes takes a lot of time

5.3 Limited Scalability for Complex Queries and Lack of User Interaction

When tackling queries with overwhelming complexity, the usability and scalability of our system are put to the test. Typically when facing queries involving multiple joins or deep nesting, our project implementation faces limitations in managing and visualising the associated QEPs effectively. As our visual output is mainly composed of static QEP tree images, users may find it hard to navigate through the cluttered information when the complexity of queries increases. Also, since our output of QEP tree is in `.jpg` format, there is no way to manipulate it after runtime and there will be a lack of user interactive features.

Future works may focus on making more interactive visualisation works such as replacing the static `.jpg` images with dynamic interactive visualisations that users can expand or collapse each node when clicked. Also, interactively moving nodes in the QEP tree to dynamically update the SQL query can also be done to add more user interactive elements into our project.